Object Oriented Programming

Chapter 6:

Method Hiding and Polymorphism

Objectives

At the end of this Lecture, students will become familiar with:

- The difference between static polymorphism and dynamic polymorphism;
- Hiding Methods;
- Hiding fields;
- Hiding nested classes;
- The difference between method hiding and method overriding.

Outline

- **Introduction**
- **Method Hiding**
- **Examples of Polymorphism**

Introduction

In C#, polymorphism allows multiple methods having different implementations to have the same name.

There are two types of polymorphism:

1. **Static or Compile Time Polymorphism**: the decision of which method to be called is made at compile time. Overloading is an example, it is the concept in which method names are the same with a different set of input parameters;

2. **Dynamic or Runtime Polymorphism**: Method overriding is an example, it can be done using inheritance. The compiler would not be aware of the method available for overriding the functionality, so the compiler does not throw an error at compile time. The compiler will decide which method to call at runtime and if no method is found then it throws an error.

Method Hiding

- When declaring a method as **virtual** in a base class, and in order to override it in a derived class, the **override** keyword should be used as a declaration modifier for the method.

- When used as a declaration modifier, the **new** keyword hides explicitly a method that is inherited from a base class. When you hide an inherited method, the derived version of the member replaces the base class version. Although you can hide public or protected members without using the new modifier, you get a compiler warning. If you use "**new**" to hide explicitly a member, the warning message disappears.

Method Hiding

```
class Rectangle {
    protected double length, width; // fields
    public Rectangle(double l, double w) { length = l;  width = w;  }
    public void Display( ) {
    Console.WriteLine("  Length: {0}, Width: {1}", length, width);}
}//end class Rectangle
class TableTop : Rectangle {
    private double cost;// price per meter square
    public TableTop(double l, double w, double c) : base(l, w) {
      cost = c;   }
    public double GetCost( ) { return length* width*cost;  }
    public void Display( ) { base.Display();
        Console.WriteLine("  Cost: {0}", GetCost());  }
}// end class TableTop
class Test  {
    static void Main( )  {
        TableTop t = new TableTop(4.5, 7.5, 100.0);
        t.Display();
    }// end Main
}// end class Test
```

The **Rectangle** class contains the **Display** method, which prints its length and width. In the derived class **TableTop**, the **Display** method prints the length, width, and cost (using **GetCost**). The **Display** method has the same signature in the two classes, but its behavior differs from one class to another. Alter running the code, the output is:

```
Length: 4.5, Width: 7.5
Cost: 3375
```

Method Hiding

The code execution was completed successfully, but the following warning message appears:

```
'TableTop.Display()' hides inherited member 'Rectangle.Display()'.
Use the new keyword if hiding was intended.
```

It means that the **TableTop.Display** method has hidden the **Rectangle.Display** method, and the **new** keyword should be used as a declaration modifier of the **TableTop.Display** method to denote the hiding process.

We add the "**new**" keyword as required:

```
new  public  void Display( ) {
             base.Display();
             Console.WriteLine("  Cost: {0}", GetCost());  }
```

When running the code again, we get the same output as before and the warning message disappears.

Method Hiding

- The **new** keyword is not only used to hide methods, it can be used to hide fields, nested classes, and other class members.

- If **new** is used with a member and there is no inheriting member with the same name, this will cause a warning message when the source code is executed.

Method Hiding

In the following code, the static field **a** is hidden:

```
1.  using System;
2.  namespace FieldHiding  {
3.      public class BaseClass     {
4.          public static int a = 15;
5.          public static int b = 30;    }
6.      public class DerivedClass : BaseClass     {
7.          // Hide field 'a'.
8.          new public static int a = 800;
9.           static void Main( )   {
10.             // Display the new value of a:
11.             Console.WriteLine(a);
12.
13.             // Display the hidden value of a:
14.             Console.WriteLine(BaseClass.a);
15.              // Display the unhidden member b:
16.             Console.WriteLine(b);
17.         }//end Main
18.     }
19. }
```

Output:

800
15
30

Method Hiding

In the following code, the class **NestedClass** is hidden:

```
1.  using System;
2.   namespace NestedClassHiding {
3.      public class BaseClass     {
4.          public class NestedClass    {
5.              public int a = 400;
6.              public int b; } } // end BaseClass
7.       public class DerivedC : BaseClass     {
8.          // Nested type hiding the base type members.
9.          new public class NestedClass       {
10.             public int a = 800, b, c; }
11.     static void Main( )    {
12.   // Creating an object from the overlapping class:
13. NestedClass nc1 = new NestedClass();
14.  // Creating an object from the hidden class:
15. BaseClass.NestedClass nc2 = new BaseClass.NestedClass();
16.              Console.WriteLine(nc1.a);
17.             Console.WriteLine(nc2.a);
18.         }//end Main
19.     }
}
```

Output:

800
400

Examples of Polymorphism

Method hiding is a dynamic polymorphism, it can be used with virtual methods. This means, we can use the keywords **new** and **virtual** together when declaring a method having a new implementation.

There are many types of static and dynamic polymorphism, but what concerns us in the context of the current presentation is to show the difference between method hiding and method overriding. Therefore, the following examples will be limited to: hiding virtual method, multiple overriding and multiple hiding.

Examples of Polymorphism: Hiding Virtual Method

```
1.  using System;
2.  class A {
3.      public virtual void F( ) {
4.          Console.WriteLine("A.F");  }
5.  }
6.  class B : A {
7.      public override void F( ) {
8.          Console.WriteLine("B.F");  }
9.  }
10. class C : B {
11.     new  public virtual void F( ) {
12.         Console.WriteLine("C.F");  }
13. }
14. class D : C {
15.     public override void F( ) {
16.         Console.WriteLine("D.F");  }
17. }
18. class Test {
19.     static void Main( ) {
20.         D d = new D();
21.         A a = d; B b = d; C c = d;
22.         a.F(); b.F(); c.F(); d.F();
23.     }
}
```

في الرمازّ الآتي، يتمّ استخدام التجاوز والإخفاء. فالطريقة العامة F موجودة في الصفوف A وB وC وD، ولها نفس التوقيع في الصفوف الأربعة، وD يرث مباشرة C الذي يرث مباشرة B، والذي بدوره يرث مباشرة A. والطريقة F افتراضية في A وC، ويتمّ تجاوزها في B وD. ويتمّ إخفاء F في C باستخدام الكلمة new، أي أنّه أصبح لـ F تنجيزٌ جديدٌ في C.

```
B.F
B.F
D.F
D.F
Press any key to continue . . .
```

Examples of Polymorphism: Multiple Overriding

```
1.  using System;
2.  class A {
3.      public virtual void F( ) {
4.          Console.WriteLine("A.F");  }
5.  }
6.  class B : A {
7.      public override void F( ) {
8.          Console.WriteLine("B.F");  }
9.  }
10. class C : B {
11.     public override void F( ) {
12.         Console.WriteLine("C.F");  }
13. }
14. class D : C {
15.     public override void F( ) {
16.         Console.WriteLine("D.F");  }
17. }
18. class Test {
19.     static void Main( ) {
20.         D d = new D();
21.         A a = d; B b = d; C c = d;
22.         a.F(); b.F(); c.F(); d.F();
23.     }
}
```

We modify the class **C** in the previous code, so that the **C.F** method has been de-hiding and is no longer a **virtual** method, but rather it is an overriding method like **B.F** and **D.F** methods. Only **A.F** is **virtual**, which is the source of the three overriding methods. After running the code, the following output is obtained:

```
D.F
D.F
D.F
D.F
Press any key to continue . . .
```

Examples of Polymorphism: Multiple Hiding

```
1.  using System;
2.  namespace HidingSequence {
3.
4.      class A {    public virtual void M() {
5.                   Console.Write("A"); }
6.              }
7.      class B : A {   public new void M() {
8.                      Console.Write("B"); }
9.              }
10.     class C : B {   public new void M() {
11.                     Console.Write("C"); }
12.         }
13.    class D : C   { public new void M() {
14.                     Console.Write("D"); }
15.         static void Main( )  {
16.             D d = new D();
17.             C c = d;     B b = c;     A a = b;
18.             a.M();   b.M();   c.M();   d.M();
19.             Console.ReadKey();
20.         } //end main }
21.     }
}
```

In this example, overriding and hiding are both used. The public method **M** is in classes: **A**, **B**, **C**, and **D**; it has the same signature in all four classes. **D** inherits directly from **C**, which inherits directly from **B**, which inherits directly from **A**. The **M** method is virtual in **A**, and is hidden in **B**, **C**, and **D** using the keyword **new**. This means that **M** has four implementations, and in each class has a different implementation. In the **Main** method, the object **d** is created from **D**, then the reference **c** is created from **C** to refer to **d**, the reference **b** is created from **B** to refer to **c**, the reference **a** is created from **A** to refer to for **b**. Then, **M** method is called by **a, b, c**, and **d**.

Examples of Polymorphism: Multiple Hiding

After running the code, we get the output:

```
ABCD
```

This is explained by the fact that for each object, the implementation found in its class is called. Each class provides independently a new implementation of the M method, so there is no need to search for other implementations.

Note: When declaring a method, the keywords **new** and **override** should not be used together.

Summary

- Static polymorphism
- Dynamic polymorphism;
- Hiding Methods;
- Hiding fields
- Hiding nested classes;
- The difference between method hiding and method overriding.

Thank you very much for attending

You can kindly ask questions